TSwap Protocol Audit Report

Version 1.0

Abdullah Amr

July 8, 2026

Prepared by: Abdullah Amr

Lead Auditor:

- Abdullah Amr

Table of Contents

Protocol Summary

TSwap is a permissionless decentralized exchange that allows users to swap assets through automated market maker pools. Instead of using an order book, the protocol uses liquidity pools containing WETH and a supported ERC20 token.

Each `TSwapPool` represents one ERC20/WETH market. Liquidity providers deposit both assets into a pool and receive LP tokens representing their share. Swappers can trade using either exact-input swaps or exact-output swaps.

The protocol is intended to maintain the constant product invariant:

```
x * y = k
```

Where:

- x is the ERC20 token reserve
- y is the WETH reserve
- k is the constant product of both reserves

Disclaimer

The Abdullah Amr team made every effort to identify as many vulnerabilities as possible within the given time period. However, this report does not guarantee that all vulnerabilities were found.

A security audit is not an endorsement of the underlying business or product. The review was time-boxed and focused only on the security aspects of the Solidity implementation of the contracts in scope.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Severity is evaluated using the CodeHawks severity matrix.

Audit Details

Scope

The following smart contracts were included in the security assessment:

```
src/PoolFactory.sol
src/TSwapPool.sol
```

Solidity Version: `0.8.20`

Target Chain: Ethereum

Tokens: Any ERC20 token

Roles

Liquidity Providers

Users who deposit WETH and a supported ERC20 token into a `TSwapPool`. In return, they receive LP tokens representing their share of the pool. Liquidity providers earn a portion of the fees generated from swaps.

Swappers / Traders

Users who interact with `swapExactInput`, `swapExactOutput`, or helper swap functions to exchange supported ERC20 tokens against WETH through a pool.

Pool Creators

Any user who calls `PoolFactory::createPool` to deploy a new `TSwapPool` for a specific ERC20/WETH pair. The protocol is permissionless, so pool creation is not restricted to an admin.

Protocol / PoolFactory

The factory contract responsible for deploying and tracking valid `TSwapPool` contracts for ERC20/WETH pairs.

TSwapPool

The AMM pool contract that holds token reserves, mints and burns LP tokens, performs swaps, and enforces the constant-product pricing logic.

Executive Summary

The audit was performed using manual review and Foundry-based testing, including unit tests, stateless fuzz testing, and stateful fuzz testing where applicable.

The review included:

- Manual inspection of the Solidity codebase
- Foundry unit testing

- Foundry fuzz testing
- Review of access control assumptions
- Review of AMM invariant assumptions
- Review of NatSpec comments and documentation

Issues Found

Severity	Number of Issues
High	5
Medium	1
Low	2
Informational	8
Total	16

Findings

High

[H-1]: `TSwapPool::deposit` does not enforce the deadline parameter, allowing stale deposits to execute

Description: The `deposit()` function accepts a `deadline` parameter. According to the function's NatSpec, this parameter is intended to define the latest time by which the transaction should be completed. However, the parameter is never checked inside the function.

As a result, a liquidity deposit can be executed even after the user-provided deadline has already passed.

Impact: A liquidity provider's transaction may execute under unfavorable market conditions even though the user supplied a deadline intended to prevent stale execution.

Proof of Concept: Add the following test to `test/unit/TSwapPool.t.sol`:

Code

```
function testDepositExecutesAfterDeadline() public {
    vm.warp(100);
```

```
3
4       uint64 expiredDeadline = uint64(block.timestamp - 1);
5
6       vm.startPrank(liquidityProvider);
7       weth.approve(address(pool), 100e18);
8       poolToken.approve(address(pool), 100e18);
9
10      // The deadline has already passed, but the deposit still succeeds.
11      pool.deposit(100e18, 100e18, 100e18, expiredDeadline);
12      vm.stopPrank();
13
14      assertGt(pool.balanceOf(liquidityProvider), 0);
15  }
```

Recommended Mitigation: Apply the existing deadline modifier to the `deposit()` function.

```
1  function deposit(
2      uint256 wethToDeposit,
3      uint256 minimumLiquidityTokensToMint,
4      uint256 maximumPoolTokensToDeposit,
5      uint64 deadline
6  )
7      external
8  +   revertIfDeadlinePassed(deadline)
9      revertIfZero(wethToDeposit)
10     returns (uint256 liquidityTokensToMint)
11 {
```

[H-2]: Incorrect fee scaling in `TSwapPool::getInputAmountBasedOnOutput` causes users to overpay for exact-output swaps

Description: The `TSwapPool::getInputAmountBasedOnOutput` function is intended to calculate how many input tokens are required to receive a specific output amount.

The protocol documentation states that swaps charge a `0.3%` fee. This should be represented using a `997 / 1000` multiplier. However, the function currently scales the numerator by `10000` instead of `1000`:

```
1  return ((inputReserves * outputAmount) * 10000) / ((outputReserves -
       outputAmount) * 997);
```

This causes users to pay approximately 10x more input tokens than required by the intended formula.

Impact: Users performing exact-output swaps are charged significantly more input tokens than expected. This results in direct user loss and incorrect AMM pricing behavior.

Proof of Concept: Add the following test to `test/unit/TSwapPool.t.sol`:

Code

```
function testGetInputAmountBasedOnOutputCalculatesIncorrectFee() public {
    vm.startPrank(liquidityProvider);
    poolToken.approve(address(pool), 100e18);
    weth.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputAmount = 1e17;
    uint256 inputReserves = poolToken.balanceOf(address(pool));
    uint256 outputReserves = weth.balanceOf(address(pool));

    uint256 expectedInputAmount =
        ((inputReserves * outputAmount) * 1000) /
        ((outputReserves - outputAmount) * 997);

    uint256 actualInputAmount = pool.getInputAmountBasedOnOutput(
        outputAmount,
        inputReserves,
        outputReserves
    );

    console.log("Expected input:", expectedInputAmount);
    console.log("Actual input:", actualInputAmount);

    assertGt(actualInputAmount, expectedInputAmount);
    assertApproxEqAbs(actualInputAmount, expectedInputAmount * 10, 10);
}
```

Recommended Mitigation: Replace `10000` with `1000`.

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
-   return ((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount) * 997);
+   return ((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount) * 997);
}
```

[H-3]: Missing `maxInputAmount` slippage protection in `TSwapPool::swapExactOutput` causes users to overpay

Description: The `swapExactOutput()` function allows a user to specify the exact amount of output tokens they want to receive. However, it does not allow the user to specify the maximum amount of input tokens they are willing to pay.

This is unlike `swapExactInput()`, which allows users to set a `minOutputAmount` to protect against unfavorable price movement.

For exact-output swaps, the equivalent slippage protection is a `maxInputAmount` parameter.

Impact: If market conditions change before the transaction is executed, the user may still receive the requested output amount but pay far more input tokens than expected. This can happen through normal price movement or through MEV/front-running.

Proof of Concept:

1. The victim wants to receive exactly 1 `WETH`.
2. Before the victim transaction executes, a whale front-runs and changes the pool price.
3. The required input amount increases.
4. The victim transaction still succeeds because there is no `maxInputAmount` check.
5. The victim receives the exact output amount but pays more input tokens than expected.

Add the following test to `test/unit/TSwapPool.t.sol`:

Code

```
function testNoSlippageProtectionInSwapExactOutput() public {
    address whale = makeAddr("whale");
    address victim = makeAddr("victim");

    vm.startPrank(liquidityProvider);
    poolToken.approve(address(pool), 100e18);
    weth.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 victimOutputAmount = 1e18;

    uint256 inputBefore = pool.getInputAmountBasedOnOutput(
        victimOutputAmount,
        poolToken.balanceOf(address(pool)),
        weth.balanceOf(address(pool))
    );

    poolToken.mint(victim, 20e18);

```

```
21      poolToken.mint(whale, 40e18);
22      vm.startPrank(whale);
23      poolToken.approve(address(pool), 40e18);
24      pool.swapExactInput(
25          poolToken,
26          40e18,
27          weth,
28          0,
29          uint64(block.timestamp + 60)
30      );
31      vm.stopPrank();
32
33      uint256 inputAfter = pool.getInputAmountBasedOnOutput(
34          victimOutputAmount,
35          poolToken.balanceOf(address(pool)),
36          weth.balanceOf(address(pool))
37      );
38
39      uint256 victimBalanceBefore = poolToken.balanceOf(victim);
40
41      vm.startPrank(victim);
42      poolToken.approve(address(pool), 20e18);
43      pool.swapExactOutput(
44          poolToken,
45          weth,
46          victimOutputAmount,
47          uint64(block.timestamp + 60)
48      );
49      vm.stopPrank();
50
51      uint256 victimBalanceAfter = poolToken.balanceOf(victim);
52      uint256 actualInputPaid = victimBalanceBefore - victimBalanceAfter;
53
54      console.log("Input needed before price move:", inputBefore);
55      console.log("Input needed after price move:", inputAfter);
56      console.log("Actual input paid by victim:", actualInputPaid);
57
58      assertGt(inputAfter, inputBefore);
59      assertEq(actualInputPaid, inputAfter);
60  }
```

Recommended Mitigation: Add a `maxInputAmount` parameter and revert if the calculated input exceeds the user's maximum acceptable input.

```
1  +error TSwapPool__InputTooHigh(uint256 inputAmount, uint256
       maxInputAmount);
2  +
3  function swapExactOutput(
4      IERC20 inputToken,
5      IERC20 outputToken,
6      uint256 outputAmount,
```

```
+       uint256 maxInputAmount,
        uint64 deadline
    )
        public
        revertIfZero(outputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 inputAmount)
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
        uint256 outputReserves = outputToken.balanceOf(address(this));

        inputAmount = getInputAmountBasedOnOutput(outputAmount,
            inputReserves, outputReserves);

+       if (inputAmount > maxInputAmount) {
+           revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount);
+       }
+
        _swap(inputToken, inputAmount, outputToken, outputAmount);
    }
```

[H-4]: `TSwapPool::sellPoolTokens` treats the user's input amount as an exact output amount

Description: The `sellPoolTokens()` function is intended to allow users to sell a specific amount of pool tokens and receive WETH in return.

The function accepts a `poolTokenAmount` parameter, which implies that the user is specifying the amount of pool tokens they want to sell. However, the implementation passes this value into `swapExactOutput()` as the `outputAmount`:

```
return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount,
    uint64(block.timestamp));
```

This means the user's intended input amount is incorrectly treated as the exact amount of WETH to receive. As a result, the function pulls however many pool tokens are required to receive that WETH amount.

Impact: Users can pay more pool tokens than intended when calling `sellPoolTokens()`. This breaks expected function behavior and can lead to user loss.

Proof of Concept: Add the following test to `test/unit/TSwapPool.t.sol`:

Code

```
function testSellPoolTokensMismatch() public {
    vm.startPrank(liquidityProvider);
```

```
3       weth.approve(address(pool), 100e18);
4       poolToken.approve(address(pool), 100e18);
5       pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6       vm.stopPrank();
7
8       uint256 userIntendedPoolTokensToSell = 1e17;
9
10      poolToken.mint(user, 10e18);
11
12      uint256 inputReserves = poolToken.balanceOf(address(pool));
13      uint256 outputReserves = weth.balanceOf(address(pool));
14
15      uint256 actualPoolTokensRequired = pool.getInputAmountBasedOnOutput
            (
16          userIntendedPoolTokensToSell,
17          inputReserves,
18          outputReserves
19      );
20
21      uint256 userBalanceBefore = poolToken.balanceOf(user);
22
23      vm.startPrank(user);
24      poolToken.approve(address(pool), 10e18);
25      pool.sellPoolTokens(userIntendedPoolTokensToSell);
26      vm.stopPrank();
27
28      uint256 userBalanceAfter = poolToken.balanceOf(user);
29      uint256 actualPoolTokensPaid = userBalanceBefore - userBalanceAfter
            ;
30
31      console.log("User intended to sell:", userIntendedPoolTokensToSell)
            ;
32      console.log("Actual poolTokens required:", actualPoolTokensRequired
            );
33      console.log("Actual poolTokens paid:", actualPoolTokensPaid);
34
35      assertGt(actualPoolTokensPaid, userIntendedPoolTokensToSell);
36      assertEq(actualPoolTokensPaid, actualPoolTokensRequired);
37  }
```

Recommended Mitigation: Use `swapExactInput()` instead of `swapExactOutput()` and allow the user to specify a minimum amount of WETH to receive. Also add a user-controlled deadline instead of using `block.timestamp` internally.

```
1  function sellPoolTokens(
2  -   uint256 poolTokenAmount
3  +   uint256 poolTokenAmount,
4  +   uint256 minWethAmountToReceive,
5  +   uint64 deadline
6  ) external returns (uint256 wethAmount) {
```

```
-    return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount,
    uint64(block.timestamp));
+    return swapExactInput(
+        i_poolToken,
+        poolTokenAmount,
+        i_wethToken,
+        minWethAmountToReceive,
+        deadline
+    );
}
```

[H-5]: Extra token incentive in `TSwapPool::_swap` breaks the constant product invariant and can drain pool reserves

Description: The protocol is intended to follow the constant product invariant:

```
x * y = k
```

However, `_swap()` contains an incentive mechanism that transfers an extra `1e18` output tokens to the caller after every `SWAP_COUNT_MAX` swaps:

```
swap_count++;
if (swap_count >= SWAP_COUNT_MAX) {
    swap_count = 0;
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
}
```

This extra transfer is not accounted for in the AMM pricing calculation. Therefore, the pool sends out more tokens than the invariant permits.

Impact: A malicious user can repeatedly perform swaps and collect the extra incentive. Over time, this can drain pool reserves and break the protocol's AMM accounting.

Proof of Concept: Add the following test to `test/unit/TSwapPool.t.sol`:

Code

```
function testInvariantBreakFromExtraSwapIncentive() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputWeth = 1e17;

    poolToken.mint(user, 100e18);

```

```
12      vm.startPrank(user);
13      poolToken.approve(address(pool), type(uint256).max);
14
15      for (uint256 i = 0; i < 9; i++) {
16          pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
                timestamp));
17      }
18
19      uint256 userWethBefore = weth.balanceOf(user);
20      uint256 poolWethBefore = weth.balanceOf(address(pool));
21
22      pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
            timestamp));
23
24      uint256 userWethAfter = weth.balanceOf(user);
25      uint256 poolWethAfter = weth.balanceOf(address(pool));
26
27      uint256 actualUserReceived = userWethAfter - userWethBefore;
28      uint256 actualPoolLoss = poolWethBefore - poolWethAfter;
29
30      assertEq(actualUserReceived, outputWeth + 1e18);
31      assertEq(actualPoolLoss, outputWeth + 1e18);
32
33      vm.stopPrank();
34  }
```

Recommended Mitigation: Remove the incentive mechanism from `_swap()`.

```
1  - swap_count++;
2  - if (swap_count >= SWAP_COUNT_MAX) {
3  -     swap_count = 0;
4  -     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
5  - }
```

Medium

[M-1]: Rebase, fee-on-transfer, and ERC777 tokens can break the protocol invariant

Description: The `_swap()` function assumes that all supported tokens strictly follow standard ERC20 behavior. Specifically, it assumes that:

- `transferFrom()` transfers exactly `inputAmount` tokens to the pool.
- `transfer()` transfers exactly `outputAmount` tokens to the recipient.
- Token balances only change as a direct result of the current swap.
- Token transfers cannot execute arbitrary external code.

These assumptions do not hold for several non-standard token implementations.

Fee-on-transfer tokens deduct a fee during transfers, causing the pool to receive fewer tokens than `inputAmount` while the protocol continues pricing as if the full amount was received.

Rebasing tokens can increase or decrease account balances asynchronously, causing the pool's actual reserves to diverge from the reserves assumed by the swap logic.

ERC777 tokens execute transfer hooks during token transfers, allowing arbitrary external code execution and potentially enabling reentrant interactions if proper protections are not in place.

Impact: Non-standard token behavior can cause reserve accounting to become inconsistent with actual balances. This may break the AMM invariant and lead to incorrect swap pricing or loss of funds.

Recommended Mitigation:

Consider one of the following approaches:

- Explicitly disallow fee-on-transfer, rebasing, and ERC777-style tokens in documentation and pool creation logic.
- Use balance-delta accounting to verify the actual amount received by the pool.
- Add reentrancy protection around state-changing swap and liquidity functions.
- Consider maintaining an allowlist of supported standard ERC20 tokens if the protocol does not intend to support arbitrary token behavior.

Low

[L-1]: `TSwapPool::LiquidityAdded` event emits parameters in the wrong order

Description: The `LiquidityAdded` event is emitted in `TSwapPool::_addLiquidityMintAndTransfer()` with the WETH and pool token amounts in the wrong order.

The event definition expects the WETH amount before the pool token amount, but the emit statement provides `poolTokensToDeposit` before `wethToDeposit`.

Impact: Off-chain indexers, analytics dashboards, and monitoring tools may read incorrect liquidity data from the emitted event.

Recommended Mitigation: Emit the event parameters in the correct order.

```
1 - emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
2 + emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2]: `TSwapPool::swapExactInput` returns the default value instead of the actual output amount

Description: The `swapExactInput()` function declares a named return value `output`, but it never assigns to it and does not use an explicit return statement.

As a result, the function always returns the default value of `0`, even though the swap may have successfully sent output tokens to the user.

Impact: Integrators and callers relying on the returned value will receive incorrect information. This can break downstream integrations or off-chain accounting.

Recommended Mitigation: Assign the calculated output amount to the named return variable.

```
function swapExactInput(
    IERC20 inputToken,
    uint256 inputAmount,
    IERC20 outputToken,
    uint256 minOutputAmount,
    uint64 deadline
)
    public
    revertIfZero(inputAmount)
    revertIfDeadlinePassed(deadline)
    returns (uint256 output)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

-   uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
+   output = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);

-   if (outputAmount < minOutputAmount) {
-       revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
+   if (output < minOutputAmount) {
+       revert TSwapPool__OutputTooLow(output, minOutputAmount);
    }

-   _swap(inputToken, inputAmount, outputToken, outputAmount);
+   _swap(inputToken, inputAmount, outputToken, output);
}
```

Informational

[I-1]: `PoolFactory::PoolFactory__PoolDoesNotExist` error is never used

Description: The `PoolFactory__PoolDoesNotExist` custom error is declared but never used.

Recommended Mitigation: Remove the unused error.

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2]: `PoolFactory::constructor` lacks a zero address check

Description: The `PoolFactory` constructor accepts a WETH token address but does not validate that it is not the zero address.

Recommended Mitigation: Add a zero address check.

```
+ error PoolFactory__ZeroAddress();
+
constructor(address wethToken) {
+   if (wethToken == address(0)) {
+       revert PoolFactory__ZeroAddress();
+   }
    i_wethToken = wethToken;
}
```

[I-3]: `PoolFactory::createPool` should use `symbol()` instead of `name()` for LP token symbol

Description: The liquidity token symbol is created using `IERC20(tokenAddress).name()` instead of `IERC20(tokenAddress).symbol()`.

Recommended Mitigation: Use `symbol()` when creating the liquidity token symbol.

```
- string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).name());
+ string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).symbol());
```

[I-4]: Prefer scientific notation for large numeric literals

Description: The codebase contains large numeric literals written in full decimal form. Using scientific notation can improve readability.

Impact: This does not affect contract security, gas consumption, or runtime behavior. It is a readability improvement.

Recommended Mitigation: Replace large numeric literals with equivalent scientific notation where appropriate.

```
// Before
uint256 constant VALUE = 1_000_000_000_000_000_000;

// After
uint256 constant VALUE = 1e18;
```

[I-5]: Unused local variable in `TSwapPool`

Description: The local variable `poolTokenReserves` is declared but not used.

```
uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

Recommended Mitigation: Remove the unused variable to improve code clarity.

[I-6]: Literal values should be replaced with named constants

Description: Several literal values are used directly in the codebase, such as fee constants and precision values. Replacing them with named constants improves readability and reduces the chance of mistakes.

Recommended Mitigation: Define constants for repeated or meaningful values.

```
uint256 private constant FEE_PRECISION = 1000;
uint256 private constant FEE_MULTIPLIER = 997;
uint256 private constant PRECISION = 1e18;
```

Then use those constants throughout the contract.

[I-7]: Solidity `0.8.20` may emit `PUSH0` opcodes

Description: Solidity `0.8.20` targets the Shanghai EVM by default. This means generated bytecode may include the `PUSH0` opcode.

Ethereum mainnet supports `PUSH0`, but some non-Shanghai-compatible EVM chains may not. If the protocol is deployed to a chain that does not support Shanghai, deployment may fail.

Recommended Mitigation: If deployment to non-Shanghai-compatible chains is intended, explicitly set the EVM version to `paris` or another compatible target.

Foundry example:

```
solc_version = "0.8.20"
evm_version = "paris"
```

[I-8]: Public function not used internally can be marked external

Description: `TSwapPool::swapExactInput()` is marked **`public`** but is not used internally.

Recommended Mitigation: If the function is not intended to be called internally, mark it as `external`.

```
- function swapExactInput(
+ function swapExactInput(
```